



AUDIT REPORT

September 2026

For

BETTER 

Table of Contents

Executive Summary	04
Number of Issues per Severity	07
Summary of Issues	08
Check Vulnerabilities	09
Techniques and Methods	10
Types of Severity	12
Severity Matrix	13
Impact	13
Likelihood	13
■ Critical Severity Issues	14
1. UnitLane lacks ERC-165, so production CCIP off-ramps skip delivery and bridged units are lost	14
■ Medium Severity Issues	15
2. Counterparty operator can pull claim-reserved cash	15
3. Zero controller creates an unserviceable redemption	16
4. Full remediation can leave pending escrow holdings	17
5. A tripped oracle is served as a fresh price through the AggregatorV3 facade	18
6. Vote signatures omit the ERC-1271 voter identity	19
■ Low Severity Issues	20
7. Redemption cancellation transfers owner-funded shares to the controller	20
8. Nominal booking can overcredit an incompatible token	21

9. Enabling a bridge before its rail cap permits unbounded minting	22
10. CCIP route rotation can strand burned units in flight	23
Functional Tests	24
Threat Model	34
1. External Dependencies & Trust Boundaries	34
1.1 External Dependencies Table	34
2. Entry & Exit Points Analysis (Function-Level)	39
2.1 Function Entry / Exit Table	39
3. Asset Flow Mapping (Critical Contracts)	64
3.1 Asset-Holding Contracts	64
3.2 Asset Entry & Exit Functions	65
Automated Tests	71
Closing Summary	72
Disclaimer	72

Executive Summary

Project Name	BXCT: BetterX tokenised fund registers and the AUDX settlement token
Protocol Type	Tokenised fund register: upgradeable ERC-4626 vaults with EIP-7540 asynchronous dealing, modular compliance and corrective powers, an oracle-priced unit register, and a fiat-backed settlement token with cross-chain supply rails.
Project URL	N/A (product is pre-launch; no public web properties are live)
Overview	The reviewed system is a bespoke, OpenZeppelin-based fund-register engine. Each fund is an upgradeable ERC-4626 vault that is simultaneously the unit token, the asset custodian and the dealing facility, assembled from discrete compliance and operating mixins that each hold their own ERC-7201 storage namespace. Unit pricing is taken from a per-fund oracle rather than an assets-over-supply ratio, and redemptions always follow the EIP-7540 asynchronous profile of request, settle and claim, with the payout locked at lodgement and the settled cash ring-fenced so that a claimable balance is never an unfunded promise. Holding and dealing are both gated by a single on-chain admission list, and the corrective powers are split so that the key able to freeze a wallet can never execute the forced move that follows it. Around the registers sit a fiat-backed settlement token with rail-attributed, rate-bounded supply, a cross-chain unit lane built as a messaging application rather than a token bridge, a KYC-gated redemption intake and a custody-free resolution ballot, with every privileged endpoint resolving through a per-fund access manager whose complete role graph the in-scope deploy kit asserts on-chain after each deployment.
Audit Scope	The scope of this audit was to analyze the BXCT smart contracts for quality, security, and correctness.
Source Code link	https://github.com/LeonieHill/BXCT/releases/tag/fullasync-v3.5-final

Contracts in Scope	src/oz/modules/RegisterCore.sol
	src/oz/modules/SyncSubscriptions.sol
	src/oz/modules/AsyncSubscriptions.sol
	src/oz/modules/AsyncRedemptions.sol
	src/oz/modules/RemediationPowers.sol
	src/oz/modules/RectificationPowers.sol
	src/oz/modules/TreasuryOps.sol
	src/oz/modules/IncomeOps.sol
	src/oz/modules/LaneOps.sol
	src/oz/modules/EligibilityPolicy.sol
	src/oz/modules/Operators7540.sol
	src/oz/RegisterVault7540.sol
	src/oz/FundVault7540.sol
	src/oz/FullAsyncVault7540.sol
	src/oz/NAVOracle.sol
	src/oz/OracleAggregatorFacade.sol
	src/oz/WhitelistRegistry.sol
	src/oz/DealingEscrow.sol
	src/oz/AsyncDepositBook.sol
	src/oz/UnitLane.sol
	src/platform/SettlementTokenCore.sol
	src/platform/SettlementToken.sol
	src/platform/CrosschainMintBurn.sol
	src/platform/CCTRegistration.sol
	src/platform/LZMinterBurner.sol
	src/platform/RedemptionRouter.sol
	src/platform/AttestationRegistry.sol
	src/platform/ResolutionBallot.sol
	src/platform/AllocationMandate.sol

Branch	main.
Commit Hash	db87d22
Language	Solidity 0.8.30
Blockchain	EVM: Ethereum (hub), Ink and Base (spokes)
Method	Manual Analysis, Threat Modelling

Review 1	27th August - 11th September, 2026
-----------------	------------------------------------

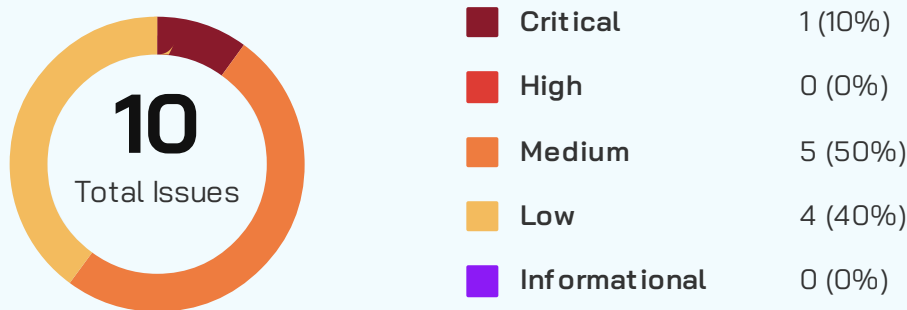
Updated Code Received	16th September, 2026
------------------------------	----------------------

Review 2	16th September, 2026
-----------------	----------------------

Fixed In	5c33ab5
-----------------	---------

✔ Verify the Authenticity of Report on QuillAudits Leaderboard: <https://www.quillaudits.com/leaderboard>

Number of Issues per Severity



		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	1	0
	Partially Resolved	0	0	0	0	0
	Resolved	1	0	5	3	0

Summary of Issues

Issue No	Issue Title	Severity	Status
1	UnitLane lacks ERC-165, so production CCIP off-ramps skip delivery and bridged units are lost	Critical	Resolved
2	Counterparty operator can pull claim-reserved cash	Medium	Resolved
3	Zero controller creates an unserviceable redemption	Medium	Resolved
4	Full remediation can leave pending escrow holdings	Medium	Resolved
5	A tripped oracle is served as a fresh price through the AggregatorV3 facade	Medium	Resolved
6	Vote signatures omit the ERC-1271 voter identity	Medium	Resolved
7	Redemption cancellation transfers owner-funded shares to the controller	Low	Resolved
8	Nominal booking can overcredit an incompatible token	Low	Resolved
9	Enabling a bridge before its rail cap permits unbounded minting	Low	Resolved
10	CCIP route rotation can strand burned units in flight	Low	Acknowledged

Check Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls

✓ Missing Zero Address Validation

✓ Private modifier

✓ Revert/require functions

✓ Multiple Sends

✓ Using suicide

✓ Using delegatecall

✓ Upgradeable safety

✓ Using throw

✓ Using inline assembly

✓ Style guide violation

✓ Unsafe type inference

✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments, match logic and expected behavior.
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper.
- Implementation of ERC standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the smart contracts.

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms used for Audit

Manual analysis, threat modelling, functional testing.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Severity Matrix

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Critical Severity Issues

1. UnitLane lacks ERC-165, so production CCIP off-ramps skip delivery and bridged units are lost

Resolved

Path	UnitLane.sol
Function name	bridge(), ccipReceive()

Description:

UnitLane and its parents do not implement supportsInterface. The CCIP off-ramp (v1.5.0, as pinned in lib/ccip, and v1.6) skips any receiver that does not advertise IAny2EVMMessageReceiver and marks the message SUCCESS, so it cannot be manually re-executed:

```
if (
  (message.data.length == 0 && message.gasLimit == 0) || message.receiver.code.length == 0
  || !message.receiver.supportsInterface(type(IAny2EVMMessageReceiver).interfaceId)
) return;
```

ccipReceive therefore never runs, while bridge has already burned the sender's units on the source chain:

```
$.vault.laneBurn(msg.sender, shareAmount);
```

Impact:

Every unit bridged through a production lane is burned and never minted on the destination, a permanent loss for every user from the first transfer.

Recommendation:

Implement ERC-165 as Chainlink's CCIPReceiver does.

Status: **Resolved**

Medium Severity Issues

2. Counterparty operator can pull claim-reserved cash

Resolved

Path	OZEstateKit.sol, AsyncSubscriptions.sol, TreasuryOps.sol
Function name	attachAsyncCounterparty(), requestDeposit(), execute

Description:

The deployment kit grants a counterparty vault unlimited settlement-asset allowance from the investing vault and grants a settlement agent ERC-7540 operator authority for that investing vault. The operator can call `counterparty.requestDeposit(x, investingVault, investingVault)` directly. The counterparty authenticates the operator and executes `transferFrom` against the investing vault. Because the call does not enter `investingVault.execute`, the investing vault's reserve postcheck is never evaluated.

If the investing vault has cash balance `B` and funded claim obligations `R`, any amount satisfying `B - R < x <= B` leaves liquid cash below its funded liabilities. With 100 units of cash and 100 units of claimable redemptions, a direct deposit of 100 leaves zero liquid cash while the 100-unit liability remains. A subsequent positive payout fails at token transfer.

Impact:

Funded investor payouts can become temporarily unavailable. Funds remain attributed to the investing vault at the counterparty. Pending cancellation may restore them; fulfilled positions require a later redemption and sufficient counterparty liquidity. Personal theft and permanent principal loss were not demonstrated. Severity reflects high payout impact and the prerequisite of a compromised or misbehaving appointed operator.

Recommendation:

Remove standing pull authority. Prefer exact temporary approvals inside a reserve-checked funding operation. Grant settlement agents a constrained claims-only capability, and ensure allowance and operator revocation remain possible during a reserve deficit.

Status: **Resolved**

3. Zero controller creates an unserviceable redemption

Resolved

Path	EligibilityPolicy.sol, AsyncRedemptions.sol
Function name	requestRedeem(), cancelRedeem()

Description:

In a permissionless profile, or when address zero is explicitly admitted, an authorized caller can submit a positive-share redemption with controller zero. Shares move into escrow, but the stored zero controller collides with the contract's unknown-request sentinel. User cancellation, governance cancellation, and fulfillment all reject the record before servicing it.

Impact:

The full affected position loses every ordinary lifecycle exit and needs exceptional governance or an upgrade. Supplied sole-subscriber mirror profiles do not admit zero and therefore block this trigger. The issue does not provide an unrelated user with access to the position.

Recommendation:

Reject `controller == address(0)` before booking a redemption. Add an explicit governance repair path for any existing zero-controller records that restores both shares and accounting.

Status: **Resolved**

4. Full remediation can leave pending escrow holdings

Resolved

Path	RemediationPowers.sol, AsyncRedemptions.sol
Function name	completeRemediation(), cancelRedeemFor()

Description:

A subject can escrow redemption shares before being frozen. Full remediation checks the subject's liquid balance and claimables, but not pending escrow shares. If governance completes remediation without first cancelling every pending redemption, the subject is unfrozen while its pending entitlement survives. The subject can then cancel and recover those shares.

Impact:

A completed whole-holding remediation can leave an arbitrarily large portion of the intended holding available to the subject. The documented `cancelRedeemFor` -then-complete workflow prevents this outcome, making the issue dependent on an operational sequencing mistake.

Recommendation:

Enforce zero pending shares as a completion precondition. Where practical, cancel pending redemptions and complete remediation atomically.

Status: **Resolved**

5. A tripped oracle is served as a fresh price through the AggregatorV3 facade

Resolved

Path	NAVOracle.sol, OracleAggregatorFacade.sol
Function name	updateRate(), latestRoundData()

Description:

updateRate halts instead of reverting on an out-of-bounds or premature post, but the store below the trip branch still runs, refreshing the timestamp and round:

```
$.rate = published;  
$.rateVersion++;  
$.lastUpdate = nowTs;
```

latestRoundData returns exactly these fields, while its NatSpec relies on a stale updatedAt as the halt signal. For example, updateRate(1) at a 1e18 rate trips, and the facade then returns answer = 1 with updatedAt = now and a new roundId, passing every standard check. Tests only cover a guardian pause(), which does not refresh lastUpdate.

Impact:

AggregatorV3 integrators price against the rejected rate as if it were fresh while the register itself refuses to deal. No integrator exists yet, which keeps this at Medium.

Recommendation:

Keep storing the tripping rate but do not advance the timestamp on a trip:

```
- $.lastUpdate = nowTs;  
+ if (!trip) $.lastUpdate = nowTs;
```

Status: Resolved

6. Vote signatures omit the ERC-1271 voter identity

Resolved

Path	ResolutionBallot.sol
Function name	castVoteBySig()

Description:

`castVoteBySig` signs only `(resolutionId, support)`. EOAs are indirectly bound by signature recovery, but ERC-1271 validation is performed against whichever voter wallet the relayer supplies. Multiple snapshot-holder wallets using a shared validator or owner set can accept the same signature and each cast a vote.

Recommendation:

Include `voter`, a nonce, and a deadline in the EIP-712 struct.

Status: **Resolved**

Low Severity Issues

7. Redemption cancellation transfers owner-funded shares to the controller

Resolved

Path	AsyncRedemptions.sol
Function name	requestRedeem()

Description:

The code comments say cancelled shares “return to the owner,” but the record does not retain the owner. `requestRedeem(shares, controller, owner)` takes shares from owner, but retains only controller in `RedeemRecord.user`.

`cancelRedeem()` authorizes that controller and transfers the escrowed shares to it. An authorized operator can lodge Alice's redemption with Bob as controller; Bob can then cancel and receive Alice's shares. Every share in a pending request can be rerouted to a distinct controller.

Recommendation:

Persist the funding owner or explicit cancellation recipient in each record and transfer cancelled escrowed shares to that address.

Status: **Resolved**

8. Nominal booking can overcredit an incompatible token

Resolved

Path	AsyncSubscriptions.sol
Function name	requestDeposit()

Description:

Deposit requests book the nominal transfer amount. A transfer-tax or negatively rebasing configured asset can therefore create liabilities exceeding custody. The supplied settlement-token implementation has neither behavior, and users cannot select the asset per request.

Recommendation:

Enforce the supported exact-transfer, non-rebasing asset profile. If broader token support is intended, account for actual receipts, outgoing transfer behavior, and rebases explicitly.

Status: **Resolved**

Client confirms this behaviour is accepted for the synchronous path, resolved for asynchronous flows.

9. Enabling a bridge before its rail cap permits unbounded minting

Resolved

Path	CrosschainMintBurn.sol, SettlementTokenCore.sol
Function name	setCrosschainBridge()

Description:

`setCrosschainBridge(bridge, true)` authorizes an ERC-7802 minting bridge without requiring a nonzero rail limit. A newly authorized bridge therefore reaches `crosschainMint` while its default cap is zero, and zero intentionally means unbounded. A later rail-limit transaction cannot undo minting in the interim.

Recommendation:

Require a nonzero `(capPerWindow, windowSecs)` before enabling a bridge, or introduce an atomic cap-and-enable operation.

Status: **Resolved**

10. CCIP route rotation can strand burned units in flight

Acknowledged

Path	UnitLane.sol
Function name	setRouter(), bridge()

Description:

`bridge` burns source units before `ccipSend`. Destination delivery accepts only the current router and current peer tuple. `setRouter` and `setPeer` overwrite these values with no legacy route window, pending-message registry, refund, or migration mechanism. A rotation between send and delivery rejects the valid old-route callback after the source burn is final. A governance or incident-response route change can strand all in-flight user transfers

Recommendation:

Version outbound route configuration, retain legacy delivery authorization until messages settle, and provide governed replay/refund recovery

Status: **Acknowledged**

Functional Tests

RegisterCore

- ✓ _update refuses a wallet to wallet transfer when the sender is not admitted
- ✓ _update refuses a wallet to wallet transfer when the receiver is not admitted
- ✓ _update refuses any transfer whose sender is frozen
- ✓ _update refuses any transfer whose receiver is frozen
- ✓ _update refuses any move touching the dealing escrow while the escrow leg is closed
- ✓ _update permits an escrow move only inside a leg the register itself opened
- ✓ mint and burn legs skip the admission branch by design, so corrective and lane legs reach unadmitted wallets
- ✓ totalAssets returns supply multiplied by the oracle rate rather than the vault asset balance
- ✓ convertToShares and convertToAssets price from the oracle rate in both rounding directions
- ✓ the rate falls back to par before an oracle is bound so ERC-4626 reads stay available
- ✓ setOracle refuses an address carrying no code
- ✓ setOracle refuses a source reporting a zero rate
- ✓ setEscrow binds once and refuses a second write
- ✓ the reserve check reverts when vault cash falls below the reserved total
- ✓ pauseDealing and pauseRedemptions are reachable by the guardian
- ✓ unpauseDealing and unpauseRedemptions are refused to the guardian and reachable by the timelock
- ✓ the upgrade guard refuses an implementation declaring a different profile identity
- ✓ the reserved storage byte in the core namespace stays zero on every reachable state
- ✓ the forced update primitive is reachable only from remediation completion and the escrow return leg

SyncSubscriptions

- ✓ deposit mints units at the published rate when no entry spread is configured
- ✓ maxDeposit returns zero while dealing is paused
- ✓ maxDeposit returns zero while the oracle is paused or tripped
- ✓ maxDeposit returns zero for a receiver who is not admitted
- ✓ maxDeposit returns zero for a frozen receiver

- ✓ maxMint returns the remaining capacity and zero once the cap is reached
- ✓ previewDeposit returns the dealing outcome net of the entry spread
- ✓ previewMint returns the gross cost including the entry spread
- ✓ the deposit and mint paths agree on the same net position under a non zero spread
- ✓ the entry spread stays in the sleeve and is emitted as accrual evidence
- ✓ the deposit hook reverts below the configured minimum subscription
- ✓ setDealingTerms refuses a spread above the estate deed cap
- ✓ initialisation refuses a deed cap above the platform ceiling
- ✓ setDealingTerms is refused on the immediate admin channel after handover
- ✓ every capacity refusal surfaces as the vendor exceeded maximum error

AsyncRedemptions

- ✓ requestRedeem parks the units on the dealing escrow and opens a pending request
- ✓ requestRedeem locks the payout at the request time rate
- ✓ requestRedeem refuses while redemptions are paused
- ✓ requestRedeem refuses while the oracle is paused or tripped
- ✓ requestRedeem refuses a size below the register minimum
- ✓ requestRedeem refuses when the dealing escrow was never bound
- ✓ requestRedeem accepts an approved operator and an independent controller
- ✓ requestRedeem consumes an ERC-20 share allowance when the caller is neither owner nor operator
- ✓ requestRedeem refuses an owner or controller who is not admitted
- ✓ requestRedeemWithPolicy refuses a discount above the register ceiling
- ✓ requestRedeemWithPolicy refuses a deadline below the register minimum
- ✓ the pending and claimable views answer only for the aggregated request identifier
- ✓ cancelRedeem is refused to every caller but the recorded controller
- ✓ cancelRedeem still works while redemptions are paused
- ✓ cancelRedeem still works after the fulfilment deadline has passed
- ✓ fulfilRedeem refuses before maturity and after the deadline
- ✓ fulfilRedeem refuses a frozen controller
- ✓ fulfilRedeem refuses a controller whose admission has lapsed

- ✓ fulfilRedeem refuses when vault cash cannot cover the reserve plus this payout
- ✓ fulfilRedeem burns the escrowed units and books a fully funded claimable
- ✓ previewRedeem and previewWithdraw revert as the standard requires
- ✓ maxRedeem and maxWithdraw return zero while the controller is frozen
- ✓ maxRedeem and maxWithdraw return zero once the controller admission has lapsed
- ✓ maxRedeem and maxWithdraw return zero while redemptions are paused
- ✓ redeem and withdraw enforce claim authority before the zero value short circuit
- ✓ a zero value claim by the controller returns zero without reverting
- ✓ a claim naming an ineligible receiver is refused even at zero value
- ✓ partial claims consume the blended locked price and cannot over draw the bucket
- ✓ claim grinding never yields more than a single full claim
- ✓ the claimable total is exported so treasury and income legs cannot spend it
- ✓ the escrow balance equals the sum of open request shares on every reachable state

AsyncSubscriptions and AsyncDepositBook

- ✓ requestDeposit pulls the subscription cash and opens a pending application
- ✓ requestDeposit refuses a zero amount
- ✓ requestDeposit refuses while dealing or the oracle is paused
- ✓ requestDeposit refuses an owner or controller who is not eligible
- ✓ requestDeposit is refused to a caller who is neither the owner nor an approved operator
- ✓ cancelDeposit refunds the owner who paid rather than the controller
- ✓ cancelDeposit refuses once the application has been struck
- ✓ cancelDeposit refuses while the owner is frozen
- ✓ fulfilDeposit strikes units at fulfilment time rather than request time
- ✓ fulfilDeposit refuses when the supplied rate version does not match the live publication
- ✓ fulfilDeposit refuses a strike that would round the unit count to zero
- ✓ fulfilDeposit refuses while the controller already holds an unclaimed strike
- ✓ claims refuse an ineligible owner, controller or receiver
- ✓ claim rounding favours the fund on both the asset and the unit denominations
- ✓ repeated partial claims cannot draw more than the struck amount

- ✓ pending and claimable subscription totals are both exported into the reserve
- ✓ disposeDeposit refuses a zero destination
- ✓ disposeDeposit refuses an empty determination reference
- ✓ the mint conduit is reachable only by the vault calling itself
- ✓ the deposit book refuses every mutator from any caller other than the bound vault
- ✓ the deposit book holds no assets and makes no external calls

RemediationPowers

- ✓ initialisation refuses a zero notice window
- ✓ the freeze refuses a zero subject
- ✓ the freeze refuses a zero destination
- ✓ the freeze refuses to overwrite the terms of a case already standing
- ✓ the freeze is reachable by the guardian and by the timelock
- ✓ completeRemediation refuses before the notice window closes
- ✓ completeRemediation refuses a partial determination while claimable cash exists
- ✓ completeRemediation sweeps claimable cash before lifting the freeze
- ✓ completeRemediation moves the holding while the subject is still frozen
- ✓ completeRemediation and the cancellation are both refused to the guardian
- ✓ every deploy asserts that the guardian holds no remediation execution capability

RectificationPowers

- ✓ rectifyMint and rectifyBurn are refused to every caller but the timelock
- ✓ rectifyMint refuses a frozen destination
- ✓ rectifyBurn refuses a frozen subject
- ✓ rectifyMint to an unadmitted subject succeeds and the units land inert
- ✓ rectification cannot touch units held by the dealing escrow
- ✓ the rectification event carries the signed unit delta and the determination reference

TreasuryOps

- ✓ execute refuses calldata shorter than a selector
- ✓ execute refuses a target that is the vault itself
- ✓ execute refuses a target and selector pair that policy has not allowlisted
- ✓ execute refuses when the pinned argument does not match the policy value
- ✓ execute refuses when the calldata is too short to contain the pinned argument
- ✓ execute bubbles the target revert reason unchanged
- ✓ execute reverts when the leg would leave the vault below its reserved cash
- ✓ setTreasuryPolicy is refused on the immediate admin channel after handover

IncomeOps

- ✓ payIncome is reachable only by the bound oracle contract
- ✓ payIncome reverts when the transfer would break the claim reserve

LaneOps

- ✓ laneBurn refuses a holder whose admission has lapsed
- ✓ laneBurn and laneMint refuse while dealing is paused
- ✓ laneMint succeeds to an unadmitted receiver and the units land inert
- ✓ crosschainBurn and crosschainMint behave identically to the lane legs
- ✓ all four legs emit the standard crosschain event twins
- ✓ all four legs are refused to any caller without the lane capability

EligibilityPolicy and WhitelistRegistry

- ✓ a zero registry admits every wallet and selects the standards only profile
- ✓ admitted returns false once a dated admission has lapsed
- ✓ admitted treats the expiry instant itself as already lapsed
- ✓ setAdmittedUntil refuses an expiry at or before the current block time
- ✓ setAdmitted clears any standing expiry date
- ✓ both admission writes ride the single allowlister capability

- ✓ the registry pointer is fixed at initialisation and exposes no setter
- ✓ a lapsed admission stops dealing and holding with no operator action required

Operators7540

- ✓ setOperator writes only for the caller own account
- ✓ an approved operator can lodge and claim for its approver
- ✓ a caller with neither operator status nor an allowance is refused

NAVOracle

- ✓ initialisation refuses a zero starting rate
- ✓ updateRate refuses a zero rate
- ✓ a post inside the bounds and after the minimum delay books income normally
- ✓ a post breaching the minimum delay halts pricing instead of reverting
- ✓ a post breaching the per post bounds halts pricing instead of reverting
- ✓ a tripping post stores the offending rate and marks the halt as a trip
- ✓ a plain unpause is refused while the halt is marked as a trip
- ✓ unpauseWithRate restates the rate and lifts the halt in one transaction
- ✓ unpauseWithRate refuses when the oracle is not paused
- ✓ the first post books nothing because the supply clock starts at zero
- ✓ income books on the smaller of the supply at the last post and the supply now
- ✓ the platform fee accrues on the conservative rate and elapsed time basis
- ✓ the fee is forfeited in full when appreciation cannot cover it
- ✓ a mark at or below par books neither a payable nor a fee
- ✓ the published rate is capped at par in par mode and floats in NAV mode
- ✓ claimYield and claimFees pay the booked amount exactly and zero the book
- ✓ claimYield and claimFees refuse a second claim and refuse while paused
- ✓ claimYield refuses an asset other than the register base
- ✓ setFeeBps refuses a value above the hard envelope
- ✓ the upgrade guard refuses an implementation of a different kind

OracleAggregatorFacade

- ✓ decimals reports eighteen and the answer mirrors the published rate
- ✓ the round identifier advances on each publication
- ✓ construction refuses a zero oracle pointer
- ✓ the description falls back when the vault exposes no symbol

UnitLane

- ✓ bridge refuses a zero recipient
- ✓ bridge refuses a destination that is not peered outbound
- ✓ bridge refuses when the quoted fee exceeds the caller maximum
- ✓ bridge burns on the source before dispatching the message
- ✓ ccipReceive refuses any caller other than the configured router
- ✓ ccipReceive refuses an unpeered source chain
- ✓ ccipReceive refuses a source lane that is not the pinned counterpart
- ✓ ccipReceive refuses a delivery key that has already been consumed
- ✓ setPeer refuses an enabled route carrying a zero counterpart lane
- ✓ setPeer refuses an outbound route carrying a zero gas budget
- ✓ setRouter refuses the zero address
- ✓ peering is refused to the bring up key once the seal has been performed

DealingEscrow

- ✓ construction refuses a zero vault
- ✓ the contract exposes no function, role, approval or upgrade path
- ✓ an unsolicited transfer into the escrow is refused by the holding rule

SettlementTokenCore and CrosschainMintBurn

- ✓ mint is refused to a caller without the minting role
- ✓ mint delivers to any destination and attributes the rail in the event log
- ✓ a rail with no configured cap mints without a quantity bound

- ✓ a configured rail refuses a mint above the remaining window allowance
- ✓ the rail window resets in full once the period has elapsed
- ✓ setRailLimit refuses a non zero cap paired with a zero window
- ✓ burn and burnFrom refuse a balance not held by a registered treasurer
- ✓ blacklisting refuses the zero address
- ✓ a blacklisted sender or receiver is refused on every transfer leg
- ✓ forceTransfer refuses a subject who is not blacklisted
- ✓ forceTransfer refuses a destination that is not a registered treasurer
- ✓ forceTransfer bypasses the blacklist gate while still respecting the pause
- ✓ crosschainMint and crosschainBurn refuse a caller without a bridge grant
- ✓ crosschainMint consumes the calling rail window allowance
- ✓ crosschainBurn destroys any holder balance with no treasurer requirement
- ✓ revoking a bridge grant kills both crosschain legs immediately
- ✓ the pause halts every transfer, mint and burn leg
- ✓ supportsInterface answers the crosschain identifier and retains the access control identifier
- ✓ the upgrade path is refused to a caller without the upgrader role
- ✓ each bespoke storage namespace resolves to its formula derived slot

RedemptionRouter

- ✓ the router is constructed paused and refuses intake until deliberately enabled
- ✓ requestRedemption refuses an amount below the configured minimum
- ✓ requestRedemption refuses a zero payout reference
- ✓ requestRedemption refuses an authorisation past its deadline
- ✓ requestRedemption refuses a payout reference that has already been consumed
- ✓ requestRedemption refuses a signature from a party without the authoriser role
- ✓ requestRedemption pulls and burns within the same call and holds nothing at rest
- ✓ revoking the authoriser role voids every outstanding authorisation at request time
- ✓ removing the router treasurer registration kills the burn leg
- ✓ the permit variant tolerates a failed permit and proceeds on a standing allowance

ResolutionBallot

- ✓ open refuses a zero document hash, snapshot root, unit total or record block
- ✓ open refuses a deadline at or before the current time
- ✓ open refuses a threshold outside the permitted band
- ✓ castVote refuses a proof that does not match the snapshot root
- ✓ castVote refuses a zero unit weight
- ✓ castVote refuses a second vote from the same holder on the same resolution
- ✓ castVote refuses a vote cast after the deadline
- ✓ castVoteBySig accepts a signature from an externally owned account
- ✓ castVoteBySig accepts a contract wallet signature through the standard checker
- ✓ close refuses before the deadline and is callable by anyone afterwards
- ✓ a resolution carries only when the threshold is met and the for weight strictly exceeds against
- ✓ cancel voids an open resolution and blocks any later close

LZMinterBurner

- ✓ construction refuses a zero token pointer
- ✓ bindAdapter is refused to every caller but the deploying binder
- ✓ bindAdapter refuses a zero adapter address
- ✓ bindAdapter refuses a second binding
- ✓ both legs are unreachable while the adapter remains unbound
- ✓ both legs are refused to any caller other than the bound adapter
- ✓ revoking the grant on the target surface evicts the peripheral entirely

AllocationMandate and AttestationRegistry

- ✓ construction refuses an empty label set
- ✓ construction refuses a label and weight length mismatch
- ✓ setMandate refuses weights that do not sum to the full basis point denominator
- ✓ setMandate refuses a weight array of a different length to the sleeve set
- ✓ attest is refused to a caller without the attestor role

- ✓ the staleness view reports true before the first attestation and after the configured age

Deploy kit: OZEstateKit and OZFullAsyncKit

- ✓ the parameter envelope refuses every out of range election
- ✓ the envelope refuses a zero governance, guardian, rate updater, solver or payout actor
- ✓ the member and permissionless profiles refuse to blur into one another
- ✓ verifyCore asserts every selector to role binding on chain after each deploy
- ✓ verifyCore asserts that the guardian holds no remediation execution capability
- ✓ verifyCore asserts the upgrade delay and the guardian veto are both configured
- ✓ handover moves the custody policy and dealing term channels onto the delayed channel
- ✓ handover grants admin to the timelock and asserts the bootstrap key is out
- ✓ sealPeering asserts the bootstrap holds neither admin nor peering afterwards
- ✓ the fully async suite refuses overlapping operational role holders
- ✓ a sealed estate can attach a new counterparty through the delayed channel with no bootstrap

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
OpenZeppelin Contracts v5.4.0	Vendored library	The entire base layer: ERC4626Upgradeable and ERC20Upgradeable for the register token, AccessManagedUpgradeable for every restricted selector, UUPSUpgradeable for the change channel, ReentrancyGuard Upgradeable, AccessManager, ERC1967Proxy, SafeERC20, Math.mulDiv for all dealing arithmetic, and the ECDSA, EIP712, MerkleProof and Signature Checker primitives used by the satellites.	The pinned tree is unmodified and behaves as audited by OpenZeppelin.	Any advisory against the pinned version applies verbatim to every register. The engine inherits the vendor's ERC-4626 rounding, AccessManager delay semantics and ERC-1967 proxy behaviour without re-deriving them.
Per-estate OpenZeppelin Access Manager	In-estate authority contract	The single capability switchboard for a fund. Holds every (target, selector) to role binding across the vault, oracle, admission registry and lane, plus role memberships, execution delays and role guardians. Every restricted modifier in the engine resolves here.	Wired correctly at deploy, asserted on-chain by the kit's verifier, and thereafter mutated only by the estate timelock.	ADMIN operations carry no target-admin delay because none is ever configured, so the holder can rebind any selector to a fresh zero-delay role and execute immediately. The two-day upgrade delay and the guardian veto therefore bind a cooperative governance executor, not the ADMIN itself.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Estate governance executor / timelock	External governance contract	Holds ADMIN, RECOVERY, REMEDIATION_EXEC, RECTIFIER and UPGRADER, and after handover also the dealing-terms, custody-policy and income-pointer channels. Executes every corrective, upgrade and recovery act.	Fully trusted per the project's stated trust model: multi-party controlled, and honouring its own scheduling delay.	A compromised executor can upgrade implementations, rebind selectors, mint or burn units by determination, relocate a frozen holding and repoint the pricing source. No on-chain control bounds it; the notice window is the only structural delay it must observe.
Guardian hot key	External signing key or Safe	Pauses dealing and redemptions on the register, pauses the pricing oracle, and serves the freeze that opens a remediation case. Also the role guardian able to cancel a scheduled upgrade.	Available, responsive and used only protectively.	One transaction halts all dealing and withholds redemption claims that are already settled and cash-covered, and only the timelock can reverse it. The same key proposes the destination and amount that the timelocked completion later executes.
Chainlink CCIP router and off-ramp	External messaging rail	UnitLane quotes the fee and dispatches unit movements through ccipSend, and accepts deliveries through ccipReceive from the configured router only. The lane operates no verification layer of its own; transport authentication is the rail's.	The rail authenticates the source lane, delivers exactly once, and invokes the receiver as configured.	The lane implements no ERC-165 interface advertisement, which production off-ramps probe before invoking a receiver. Router rebinding rides the upgrade channel. On a relayer-backed testnet rail the relayer supplies the whole message and is a fully trusted mint authority.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
AUDX settlement token	In-estate ERC-20, separate role graph	The register's asset(). Every subscription, redemption payout, income distribution, fee claim and treasury movement is an AUDX transfer, and each register's cash position is a balance on this contract.	Solvent, unpaused, and no register address is ever screened out.	A global pause or a blacklist entry naming a vault halts every cash leg of every fund while unit accounting keeps running. The screening and salvage roles are held by the same key in the deployed rail, so one address can screen a vault and then move its balance.
AUDX reserve backing	Off-chain issuer obligation	AUDX is issued against fiat reserves the issuer holds. The fund registers price and settle entirely in it, and nothing in the vault stack reads a reserve attestation.	The issuer holds full backing and honours redemption at par.	A shortfall or depeg is invisible to every contract in scope. The register would continue to deal at a par that no longer reflects the settlement asset's real value.
Rate-updater bot	Off-chain semi-trusted role	Holds the pricing capability. Publishes the dealing rate that drives every ERC-4626 conversion, every redemption price lock and every subscription strike, and transcribes external-position valuations onto the mirror registers.	Posts the fund accountant's struck price, within the configured per-post bounds and cadence.	A single out-of-bounds post halts dealing and stores the offending rate, with recovery available only to the timelock. Silence leaves a stale rate in place: the engine applies no staleness gate on the consumption side.
Settlement agent / solver bot	Off-chain semi-trusted role	Holds the settlement capability for both dealing sides and the treasury capability for policy-scoped custody calls. Chooses which lodged requests to settle and when, and deploys or recalls the fund's sleeve.	Settles lodged requests within the disclosed window and moves the sleeve only within the allowlisted policy.	Settlement is discretionary: there is no queue, no ordering rule and no obligation to act before a request expires. On feeder estates the counterparty's equivalent key is additionally an approved operator of the estate on the parent register.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Allowlister / compliance system	Off-chain semi-trusted role	Writes admission and dated admission on the registry. This list is the entire holding rule: it gates wallet-to-wallet transfers, subscription, redemption lodgement, settlement and payout.	Reflects the operator's verified compliance record and writes real credential expiries.	A mis-set expiry silently de-admits a holder mid-lifecycle and strands their settled claim while the cash stays reserved. Per-chain registries are independent, so a de-admission on one venue does not propagate to another.
Platform authorisation key	Off-chain signer	Signs the typed redemption authorisation the intake router requires, after identity and bank-account verification. Without a live signature the router accepts nothing.	Signs only for verified holders against a real payout instruction.	A leaked authorisation stays valid until its own deadline. Revoking the role voids outstanding signatures, since authorisation is checked at request time rather than at signing time.
LayerZero endpoint and adapter	External transport, ships disabled	The peripheral translates LayerZero's mint and burn dialect onto the ERC-7802 surface of either the settlement token or a unit register. No endpoint, adapter or verifier configuration exists in this release.	Enabling it later is a timelocked configuration ceremony, not a code change.	Once bound, the adapter is an unbounded mint and burn authority on whichever ERC-7802 surface it was pointed at, revocable only by withdrawing that surface's grant.
Canonical CREATE2 deployer	External infrastructure	Every implementation and every proxy is deployed through the canonical deterministic-deployment factory at fixed salts, giving one reviewed bytecode per chain and a derivable address per fund.	The canonical factory is present and unmodified on every target venue.	A missing or non-canonical factory on a new chain changes every derived address in the estate. The kit asserts that code landed at the expected address but does not assert the factory's own identity.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Off-chain reconciliation and attestation	Off-chain process	The attestation registry anchors the hash of the operator's daily reconciliation report, and the mirror registers' published valuations are transcriptions of real bank positions.	The operator reconciles honestly and transcribes accurately.	No contract verifies that a mirror's posted valuation matches a real balance, or that units burned on one venue were minted on another. Cross-chain supply conservation and reserve adequacy are both off-chain assurances.

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Contract Name	Function	Category
FundVault7540	deposit() / mint()	What this function can do Pull settlement assets from the caller, mint units to the receiver at the published dealing rate, retain any configured entry levy inside the sleeve, and emit that levy as evidence for the next valuation strike. What this function cannot/should not do It must not issue units while dealing or pricing is halted, to an unadmitted or frozen receiver, or beyond the configured capacity cap. Every such refusal surfaces as the vendor's own ExceededMax error rather than a bespoke revert. Main invariant(s) Units minted equal the net subscription converted at the published rate. The entry levy never leaves the vault and accrues to the fund; there is deliberately no beneficiary parameter. Access level Permissionless. Every gate is expressed through maxDeposit and maxMint returning zero, so the vendor deposit flow is used verbatim.
FundVault7540	transfer() / transferFrom()	What this function can do Move units between two wallets that are both currently admitted and neither frozen. What this function cannot/should not do It must not move units to or from the dealing escrow outside a leg the register itself opened, and it must not move units to or from any frozen wallet or any wallet that is not admitted. Main invariant(s) The single-allowlist holding rule holds on every wallet-to-wallet leg. Total supply is unchanged and the escrow is unreachable by any unsolicited transfer. Access level Permissionless, subject to the holding rule.

Contract Name	Function	Category
FundVault7540	requestRedeem()	What this function can do Assume control of the owner's units, park them on the dealing escrow, lock the payout at the current rate, and open a pending request controlled by a nominated party. What this function cannot/should not do It must not lock a price while the oracle is paused or tripped, accept a size below the register minimum, or proceed on a register whose escrow was never bound. Main invariant(s) Escrowed units equal the sum of open request shares at all times. The locked payout is fixed at lodgement and is never repriced by later rate posts. Access level Permissionless for the owner, and also callable by an approved operator or by a holder of an ERC-20 share allowance. Either may nominate any admitted wallet as controller.
FundVault7540	requestRedeemWithPolicy()	What this function can do Lodge the caller's own exit with a chosen early-exit discount and a chosen settlement deadline, returning the per-request identifier directly to the caller. What this function cannot/should not do It must not accept a discount above the register's ceiling or a deadline below the disclosed backstop, and it must not act on behalf of a third party: owner and controller are both forced to the caller. Main invariant(s) The discount is bounded by the per-register ceiling and the deadline is floored at the register minimum. No ceiling is applied to the chosen deadline beyond the parameter's own type width. Access level Permissionless and caller-scoped. This is also the exit selector the deploy kit allowlists as a feeder treasury leg, precisely because it carries no address argument to constrain.

Contract Name	Function	Category
FundVault7540	cancelRedeem()	What this function can do Withdraw an open pending request in full and return the escrowed units. What this function cannot/should not do It must not be callable by anyone other than the request's recorded controller, and it must not act on a request that has already been settled or never existed. Main invariant(s) The record is deleted and the pending total decremented by exactly the escrowed amount, so the escrow balance identity is preserved. Access level Permissionless for the recorded controller. Deliberately unaffected by the redemption pause and by the settlement deadline, so units can never be trapped by an operational halt.
FundVault7540	redeem() / withdraw()	What this function can do Claim a settled redemption, consuming the controller's aggregated claimable balance at its blended locked price and paying the settlement asset to any currently eligible receiver. What this function cannot/should not do It must not pay a frozen or de-admitted controller, pay while the redemption facility is halted, or allow any partition of partial claims to consume or pay more than the bucket holds. Main invariant(s) Both sides of the claimable bucket are decrement-bounded, so over-claiming is impossible and each rounding direction favours the fund. Access level Permissionless for the controller or an approved operator. The ERC-20 allowance path authorises requests only and is never accepted for a claim.

Contract Name	Function	Category
FundVault7540	setOperator()	What this function can do Grant or revoke operator status over the caller's own account. What this function cannot/should not do It must not be settable for another party's account. Main invariant(s) Self-scoped by construction, since the mapping is keyed on the caller. Access level Permissionless and self-scoped. The grant conveys full request authority, including the ability to nominate a different controller for the caller's units.
FundVault7540	fulfilRedeem()	What this function can do Settle one matured, unexpired request by burning the escrowed units and booking the locked payout as a funded claimable balance for the controller. What this function cannot/should not do It must not settle for a frozen or de-admitted controller, settle outside the maturity and deadline window, or settle unless the vault already holds every outstanding reserved liability plus this payout. Main invariant(s) A claimable balance is fully cash-covered at the moment it is created. Supply falls by exactly the escrowed units and the reserve rises by exactly the locked payout. Access level Settlement capability only. Settlement is discretionary: the function acts on one named request with no queue, no ordering rule and no obligation to act before expiry.

Contract Name	Function	Category
FundVault7540	cancelRedeemFor()	What this function can do Return a frozen controller's escrowed units into their own still-frozen balance, so a subsequent completion can sweep the whole holding in one move. What this function cannot/should not do It must not touch a request whose controller is not frozen. This is the freeze and escrow closure, never a general power to cancel another party's request. Main invariant(s) Supply-preserving. The holding-rule bypass is used exactly once, with both endpoints already proven non-zero by the request's own existence check and the one-shot escrow binding. Access level Remediation execution capability, held by the timelock alone. The key that can freeze a wallet can never reach this.
FundVault7540	freezeSharesAndStartRemediation()	What this function can do Immobilise a wallet immediately and record the determination's destination, amount and reference together with the expiry of the notice period. What this function cannot/should not do It must not accept a zero subject or a zero destination, and it must not overwrite the terms of a case that is already standing. Main invariant(s) The notice binds its own terms: changing destination, amount or clock requires an explicit governance cancellation first, so a hot key can never silently substitute what the timelocked completion will execute. Access level Remediation capability, held by the guardian hot key and by the timelock. Protective and immediate by design.

Contract Name	Function	Category
FundVault7540	completeRemediation()	What this function can do After the notice period, redirect the subject's unclaimed redemption cash to the determination address and move their holding there as a single supply-preserving transfer, then lift the freeze. What this function cannot/should not do It must not execute before the notice period closes, and it must not proceed on a partial determination while unclaimed cash exists, since that would either seize beyond scope or leak value on unfreeze. Main invariant(s) The move executes while the subject is still frozen, so no instant exists in which they are unfrozen while still holding units, and the evidence is one transfer rather than a supply dip and restore. Access level Remediation execution capability, held by the timelock alone.
FundVault7540	cancelRemediationAndUnlockShares()	What this function can do Void a standing notice and release the subject's wallet without moving anything. What this function cannot/should not do It must not act where no notice is standing. Main invariant(s) Deleting the case record is the only route to an unfreeze other than a completed determination. Access level Remediation execution capability, held by the timelock alone.

Contract Name	Function	Category
FundVault7540	rectifyMint() / rectifyBurn()	What this function can do Create or destroy units under a written determination to correct the register, whether for error correction, compliance with an order, or reconciliation. What this function cannot/should not do Neither must touch a frozen account or the dealing escrow. Relocating a frozen holding is the remediation lane's exclusive function, and it carries a notice period that rectification does not. Main invariant(s) Corrective legs run through the ordinary mint and burn path, so the freeze test and the escrow gate both still apply and the vendor's zero-address guards remain in force. Access level Rectification capability, held by the timelock alone. The determination reference is recorded in the event but is not validated as non-zero, and no amount bound applies.
FundVault7540	execute()	What this function can do Make one policy-allowlisted call from the vault to a named target, optionally requiring that one static address argument matches a pinned value, then re-verify the reserve. What this function cannot/should not do It must not call the vault itself, dispatch a target and selector pair that policy has not allowlisted, or leave the vault holding less than its reserved liabilities. Main invariant(s) Treasury legs can never spend claim-reserved cash. The prohibition on calling the vault itself is what keeps the asynchronous mint conduit closed. Access level Treasury capability. Policy is written by the administrator during bring-up and by the delayed, guardian-vetoed channel after handover.

Contract Name	Function	Category
FundVault7540	setTreasuryPolicy()	What this function can do Bind or clear one custody leg and pin one static address argument within it. What this function cannot/should not do It must not remain on the immediate administrative channel after handover, since rewriting the allowlist that scopes the hot treasury key is itself a change-control act. Main invariant(s) Only a single address argument may be pinned per leg. Value arguments such as amounts and discounts cannot be constrained by the policy shape at all. Access level Administrator during bring-up, then the delayed upgrade channel with the guardian able to veto.
FundVault7540	payIncome()	What this function can do Transfer booked distribution or fee cash out of the vault's sleeve to the address the oracle names, then re-verify the reserve. What this function cannot/should not do It must not be reachable by anything other than the register's own bound oracle, and it must not leave the vault short of its reserved claim cash. Main invariant(s) Income legs preserve the claim-funding reserve on exactly the same terms as treasury legs. Access level Income-payer capability, granted only to the register's own pricing oracle contract, never to a key.

Contract Name	Function	Category
FundVault7540	laneBurn() / crosschainBurn()	What this function can do Destroy units at a named holder as the source half of a cross-chain movement, and emit the standard cross-chain evidence twin. What this function cannot/should not do Neither must burn from a holder whose admission has lapsed, and neither must run while dealing is halted. Main invariant(s) The explicit admission test here is load-bearing, because the core holding rule does not test admission on a burn leg. Without it a de-admitted holder could exfiltrate units to an admitted party on another venue. Access level Lane capability, held by the lane proxy. The parameter accepts an arbitrary holder and an unbounded amount, with no rate window of the kind the settlement token applies to its own rails.
FundVault7540	laneMint() / crosschainMint()	What this function can do Create units at the destination receiver as the second half of a cross-chain movement. What this function cannot/should not do Neither must run while dealing is halted, and neither must mint into a frozen wallet. Main invariant(s) Admission is deliberately not required. Bridged units land inert under the holding rule and cannot transfer, redeem or bridge onward until the receiver is admitted. Access level Lane capability. These legs bypass the subscription capacity cap, which is enforced only through the deposit limits.

Contract Name	Function	Category
FundVault7540	pauseDealing() / pause Redemptions()	What this function can do Halt subscriptions and lane movement, or halt redemption settlement and claims, in a single transaction. What this function cannot/should not do Neither must be reversible by the same key that pulled it. Main invariant(s) The pause and recovery asymmetry is expressed as two distinct capabilities rather than one. Access level Guardian hot key and the timelock. Note that halting redemptions also withholds claims that are already settled and cash-covered.
FundVault7540	unpauseDealing() / unpause Redemptions()	What this function can do Restore a halted facility. What this function cannot/should not do Neither must ever be reachable by the guardian hot key. Main invariant(s) Recovery is a governance act, never an operational one. Access level Recovery capability, held by the timelock alone.

Contract Name	Function	Category
Sync Subscriptions	setDealingTerms()	What this function can do Set the minimum subscription size, the entry levy and the capacity cap for the register. What this function cannot/should not do It must not set a levy above the ceiling pinned into the estate at initialisation, which itself can never exceed the platform-wide constant. Main invariant(s) A three-tier bound applies: an immutable platform constant, a per-fund ceiling fixed at deployment with no setter, and the live dial beneath it. Access level Administrator during bring-up, then the delayed, guardian-vetoed channel, because these are disclosed product terms rather than operational knobs.
RegisterCore	setOracle() / setEscrow()	What this function can do Bind the register's pricing source, and bind the dealing escrow exactly once. What this function cannot/should not do The escrow must not be re-bound once set. The oracle must not accept an address carrying no code or a source reporting a zero rate. Main invariant(s) The escrow binding is one-shot and permanent. The oracle binding remains changeable but seasons behind the upgrade channel, because swapping the pricing source reprices the register as surely as swapping the implementation. Access level Escrow binding is administrative. Oracle binding rides the delayed upgrade channel alongside implementation changes.

Contract Name	Function	Category
RegisterCore	upgradeToAndCall()	What this function can do Replace the register implementation behind the proxy. What this function cannot/should not do It must not accept an implementation declaring a different profile identity, since the two product compositions share a proxy identifier and an unguarded swap could silently change the dealing profile. Main invariant(s) Upgrades are same-profile only. Storage-layout compatibility remains a separate procedural release gate that this check cannot enforce. Access level Upgrade capability, held by the timelock with a two-day execution delay and the guardian able to cancel within the window.
FullAsync Vault7540	requestDeposit()	What this function can do Pull the owner's subscription money into the vault and open a pending application under a stable identifier, controlled by a nominated party. What this function cannot/should not do It must not accept a zero amount, run while dealing or pricing is halted, or name an owner or controller who is not currently eligible. Main invariant(s) Pending subscription money is reserved from this instant: the fund cannot deploy it, income legs cannot spend it, and it is refundable in full. Access level Permissionless for the owner, and also callable by an approved operator, who additionally requires the owner's standing settlement-asset approval to the vault.

Contract Name	Function	Category
FullAsync Vault7540	cancelDeposit()	What this function can do Withdraw a pending application and return the full subscription amount to the owner who paid it. What this function cannot/should not do It must not act on an application that has already been struck, and it must not proceed while the owner is frozen. Main invariant(s) The refund goes to the payer rather than the controller, because the deposit ledger retains owner attribution that the redemption record does not. Access level Permissionless for the owner, the controller, or an approved operator of either.
FullAsync Vault7540	fulfilDeposit()	What this function can do Strike a pending application at the live rate under an explicit publication pin, recording the units owed together with the dealing-window and rate-version evidence. What this function cannot/should not do It must not strike while dealing or pricing is halted, against a different publication than the settlement agent named, for an ineligible owner or controller, or where the resulting unit count would round to zero. Main invariant(s) Reserve-neutral: pending money simply becomes claimable money and the total reserved figure is unchanged. No units are created at this point. Access level Settlement capability. Refuses while the controller already holds an unclaimed strike, so applications serialise one at a time per controller.

Contract Name	Function	Category
FullAsync Vault7540	deposit() / mint() (claim)	What this function can do Consume part or all of the controller's struck application and mint the corresponding units to any currently eligible receiver. What this function cannot/should not do It must not mint for an ineligible controller, owner or receiver, run while dealing is halted, or draw more than the open claim still owes. Main invariant(s) This is the moment reserved trust money becomes fund property. Both rounding directions favour the fund and the ledger's decrement bounds make over-claiming impossible. Access level Permissionless for the controller or an approved operator.
FullAsync Vault7540	disposeDeposit()	What this function can do Seize a pending or struck application under a determination, send the money to the named destination, and clear the controller's open claim. What this function cannot/should not do It must not execute without a non-zero destination and a non-zero determination reference, and it must not act on a closed record. Main invariant(s) This is the only corrective power in the estate that refuses to execute unevicenced, and it is also the sole route to clear a claim slot the controller has wedged. Access level Remediation execution capability, held by the timelock alone.

Contract Name	Function	Category
FullAsync Vault7540	asyncDeposit Mint()	What this function can do Mint struck units to the receiver a claim names. What this function cannot/should not do It must not be reachable by any caller other than the vault itself. Main invariant(s) Guarded solely by the requirement that the caller is the vault. The treasury conduit's refusal to call the vault itself is what keeps that guard closed. Access level Self-call conduit only, reached from the delegatecall extension executing in the vault's own context.
NAVOracle	updateRate()	What this function can do Publish a new unit price, book the distribution payable and the accrued platform fee on a conservative supply basis, and advance the publication identity. What this function cannot/should not do It must not accept a zero rate under any circumstance. A post breaching the minimum delay or the per-post bounds must halt pricing protectively rather than be accepted silently. Main invariant(s) Income books on the smaller of the supply at the previous post and now, so dealing between posts cannot manufacture income, and the fee never consumes principal when appreciation cannot cover it. Access level Pricing capability, held by the rate-updater bot. A breaching post stores the offending rate, marks the halt as a trip, and blocks a plain recovery.

Contract Name	Function	Category
NAVOracle	claimYield() / claimFees()	What this function can do Pull the accrued distribution payable or platform fee from the vault's sleeve to the configured payout address and zero the corresponding book. What this function cannot/should not do Neither must run while pricing is halted, and neither must be invoked naming an asset other than the register's own base. Main invariant(s) Each claim pays exactly the booked amount once, and the vault's reserve check still governs the transfer, so an unfunded distribution reverts rather than dipping into claim cash. Access level Distribution capability for the yield leg; the timelock for the fee leg.
NAVOracle	pause() / unpause() / unpauseWithRate()	What this function can do Halt pricing protectively, lift a plain halt, or atomically restate a governance-determined rate and lift a tripped halt in a single transaction. What this function cannot/should not do A plain recovery must refuse while the halt was caused by a breaching post, since the offending rate is still stored and lifting the halt would reopen dealing at that price. Main invariant(s) Trip recovery re-bases the delay and bounds clock, so the next ordinary post is measured against the restated rate rather than the rejected one. Access level Guardian for the halt; recovery capability held by the timelock for both restoration paths.

Contract Name	Function	Category
Whitelist Registry	setAdmitted() / setAdmitted Until()	What this function can do Grant or revoke undated admission, or grant admission that lapses on-chain at a stated expiry, and clear any standing date when the boolean is written. What this function cannot/should not do A dated admission must not be written with an expiry at or before the current block time. Main invariant(s) One list gates both dealing and holding. The expiry is a strict bound, so the block whose timestamp equals it is already lapsed, and the chain learns when an admission ends but never what ended. Access level Admission capability on member estates, or the timelock where the deployment elects to run without a standing admission key. The registry pointer itself is fixed at initialisation and has no setter.
UnitLane	bridge()	What this function can do Burn the caller's units on this venue, pull and forward the rail fee, and dispatch a message instructing the peer lane to mint the same quantity to a named recipient. What this function cannot/should not do It must not accept a zero recipient, dispatch toward a venue that is not peered outbound, or proceed when the quoted fee exceeds the caller's stated maximum. Main invariant(s) The burn and the dispatch are atomic, so a failed send unwinds the burn. Supply is conserved across venues only if the destination leg actually executes. Access level Permissionless for any admitted holder; the admission test is applied by the vault's burn leg rather than here.

Contract Name	Function	Category
UnitLane	ccipReceive()	What this function can do Accept a delivered message and mint the bridged units to the receiver it names. What this function cannot/should not do It must not accept delivery from any address other than the configured router, from an unpeered source venue, from a source lane other than the pinned counterpart, or consume the same delivery key twice. Main invariant(s) Exactly-once consumption is enforced locally by a domain-separated delivery key, independent of whatever guarantee the underlying rail provides. Access level The configured router only. The contract advertises no interface identifier, which production off-ramps probe before invoking a receiver.
UnitLane	setPeer() / setRouter()	What this function can do Enable or disable a route in either direction with a pinned counterpart lane and destination gas budget, or rebind the delivery router. What this function cannot/should not do An enabled route must not carry a zero counterpart lane, and an outbound route must not carry a zero gas budget, since either would strand burned units behind an undeliverable message. Main invariant(s) Peering is two-sided and independently configured, so a half-completed ceremony burns on the source and never mints on the destination. Access level Peering capability, held by the bring-up key and renounced at the seal. Router rebinding rides the delayed upgrade channel.

Contract Name	Function	Category
AsyncDeposit Book	request() / fulfil() / claim() / cancel() / dispose() / remediate Claimable()	What this function can do Record and mutate every asynchronous subscription: owner and controller attribution, request state, per-party and register-wide totals, the single open claim per controller, and the dealing-window evidence. What this function cannot/should not do No caller other than the bound vault must reach any of them, and no transition must run from the wrong state or clear a claim that is not the current open one. Main invariant(s) A pure ledger. It custodies nothing, makes no external calls, and every aggregate moves in lockstep with the underlying records. Access level Vault-only. The binding is immutable and fixed at construction, and the vault verifies it points back before accepting the book.
SettlementToken Core	mint()	What this function can do Create settlement tokens at any destination and attribute the supply change to the calling rail in the event log. What this function cannot/should not do It must not exceed the calling rail's remaining window allowance, deliver to a screened address, or run while the token is paused. Main invariant(s) Every supply increase names its authority. A rail with no configured cap is unbounded by election, and the window resets wholesale rather than rolling. Access level Minting role. Destinations are deliberately open, matching the retail-circulation posture of comparable fiat-backed tokens.

Contract Name	Function	Category
SettlementToken Core	burn() / burnFrom()	What this function can do Destroy settlement tokens held at a registered treasurer wallet as an issuer ramp operation. What this function cannot/should not do Neither must destroy a balance that does not sit on a registered treasurer, since a holder-side burn would detach circulating supply from the reserve while the issuer still holds the backing. Main invariant(s) Supply contracts only at wallets the issuer has registered, which is what keeps the reserve attestation meaningful. Access level Open in form, but the wallet whose balance is destroyed must be a registered treasurer, which is the structural constraint.
CrosschainMint Burn	crosschainMint() / crosschain Burn()	What this function can do Create or destroy settlement tokens at any holder through the standard cross-chain surface, emitting both the standard twin and the rail-attribution event. What this function cannot/should not do Neither must run for a caller without an explicit bridge grant, and the mint must not exceed the rail's window allowance. Main invariant(s) The mint side is rate-bounded and screened. The burn side carries neither a window nor a treasurer requirement, so an authorised bridge can destroy any holder's balance. Access level An explicit, revocable grant per bridge. The authority registry is the access manager itself rather than a separate list.

Contract Name	Function	Category
SettlementToken Core	forceTransfer()	What this function can do Move a screened holder's balance to a registered treasurer under a recorded determination. What this function cannot/should not do It must not act on a holder who is not screened, deliver anywhere other than a treasurer, or run while the token is paused. Main invariant(s) It deliberately bypasses the screening gate while still respecting the pause, because the salvage power exists precisely to move value off a screened account. Access level Salvage role. In the deployed rail this is the same key that holds the screening role, so one address can both screen an account and then move its balance.
SettlementToken Core	addToBlacklist() / removeFromBlacklist()	What this function can do Screen an address out of every transfer leg, or restore it. What this function cannot/should not do It must not screen the zero address, which participates in every mint and burn leg and would brick issuance and redemption for the whole rail. Main invariant(s) Screening applies to both endpoints of every transfer, including the standard cross-chain legs. Access level Screening role. Screening a register's vault address halts every cash leg of that fund while its unit accounting continues to run.

Contract Name	Function	Category
SettlementToken Core	addTreasurer() / removeTreasurer() / setRailLimit() / setCrosschain Bridge()	What this function can do Register or evict the issuer's burn-boundary wallets, bind a rail's window cap, and authorise or revoke a bridge's cross-chain access. What this function cannot/should not do A non-zero cap must not be bound without a non-zero window period. Main invariant(s) Every transport's authority is an explicit revocable grant, and removing a treasurer registration is the structural kill switch for any burn path that depends on it. Access level Default administrator role on the settlement token, which sits outside the fund estates' access managers.
SettlementToken Core	pause() / unpause()	What this function can do Halt or restore every transfer, mint and burn leg on the settlement token. What this function cannot/should not do Nothing is exempt: the salvage power respects the pause, as do both cross-chain legs. Main invariant(s) A pause stops every cash leg of every register that deals in this asset, while those registers' unit accounting continues unaffected. Access level Pauser role. Both directions sit with the same role, unlike the registers' deliberate split between a hot pause key and timelocked recovery.

Contract Name	Function	Category
Redemption Router	request Redemption() / requestRedemption WithPermit()	What this function can do Accept an issuer-signed authorisation, pull the stated amount from the holder, burn it in the same call, and emit the payout instruction the off-chain run settles against. What this function cannot/should not do It must not accept intake while paused, below the disclosed minimum, past the authorisation deadline, with a payout reference already consumed, or with a signature from anyone but a live authoriser. Main invariant(s) One authorisation, one burn: the payout reference is marked consumed before any token movement, and no balance is held at rest. Access level Open in form but unusable without a live issuer signature. Ships constructed paused, with three independent cut-offs available.
Resolution Ballot	open() / cancel() / close()	What this function can do Open a resolution against a register snapshot with a stated threshold and deadline, void one for procedural defect, or close one after its deadline and record the outcome. What this function cannot/should not do A resolution must not open with a zero document hash, zero root, zero units, a past deadline or a threshold outside the permitted band, and it must not close before its deadline. Main invariant(s) The threshold applies to the votes actually cast rather than to units on issue, so no minimum turnout is required for a resolution to carry. Access level Opening and cancelling are owner-only, held by the estate's governance executor. Closing is permissionless once the deadline has passed.

Contract Name	Function	Category
Resolution Ballot	castVote() / cast VoteBySig()	What this function can do Record a unit-weighted vote proven against the register snapshot, either directly from the holder's wallet or through a relayed typed signature that contract wallets can also produce. What this function cannot/should not do Neither must accept a second vote from the same holder on the same resolution, a vote cast after the deadline, or entitlement that the merkle proof does not establish. Main invariant(s) Entitlement derives from the snapshot proof rather than a live balance, and the ballot holds no assets and executes nothing on the register. Access level Permissionless with a valid proof. Relayed signatures carry no nonce, no expiry of their own and no revocation path, so one remains usable until the resolution closes.
LZMinterBurner	bindAdapter() / mint() / burn()	What this function can do Bind the transport adapter exactly once after both sides exist, then translate that transport's mint and burn dialect onto the cross-chain surface it was constructed against. What this function cannot/should not do The binding must not be performed twice, by anyone but the deploying key, or to the zero address, and neither leg must be callable by anything other than the bound adapter. Main invariant(s) Until bound, both legs are unreachable because no caller can be the zero address. All policy lives on the surface it points at, never here. Access level Deploy-time binder for the one-shot binding; the bound adapter alone thereafter. Ships unbound and is evicted by revoking the grant on the target surface.

Contract Name	Function	Category
Attestation Registry / Allocation Mandate	attest() / setMandate()	What this function can do Anchor the hash of the operator's reconciliation report on-chain with a timestamp, and replace a fund's target sleeve weights. What this function cannot/should not do Mandate weights must not be accepted unless they sum to exactly the full basis-point denominator, and the sleeve labels themselves must not change. Main invariant(s) Both are records rather than controls: neither holds assets, grants powers, nor is read by any dealing path, so divergence between them and reality is an operational finding rather than a contract state. Access level Attestor role for the report anchor. Mandate weights sit with a plain single-step ownership model outside the estate's access manager.

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
FundVault7540	Settlement asset (ERC-20) and its own unit supply	The fund's entire settlement-asset sleeve: unspent subscription cash, the redemption liquidity buffer, and the ring-fenced reserve backing every settled but unclaimed redemption. It is simultaneously the unit register itself, minting and burning the token that represents the holding.
FullAsyncVault7540	Settlement asset (ERC-20) and its own unit supply	The same custody as the synchronous register, plus subscription money held from lodgement until the claim converts it into units. Pending and struck subscription cash is reserved and refundable, and is not the fund's property until claimed.
DealingEscrow	Fund units (the register's own ERC-20)	Every unit parked in an open redemption request. It holds no settlement asset and exposes no functions, roles, approvals or upgrade path; the vault moves units in and out through its own token accounting.
SettlementToken	Its own ERC-20 supply	Not a custodian of third-party assets so much as the asset itself. Every register's cash position is a balance on this contract, so its pause and screening state gates every cash leg in the estate.
RedemptionRouter	Settlement asset (transient)	Holds no balance at rest. Tokens transit only inside a single intake call, pulled from the holder and burned before the call returns. Registered as a treasurer purely so that burn leg is permitted.

Contract	Asset Type	Custodied Assets
UnitLane	Rail fee token (transient)	Holds no balance at rest. The bridge fee is pulled from the caller, approved to the router and consumed within the same transaction, so no residue is expected and no sweep function exists.
AsyncDepositBook	None	A pure ledger for the asynchronous subscription flow. It records attribution, state, totals and dealing evidence while the vault remains sole custodian, and it does not import a token interface at all.
NAVOracle	None (holds a claim, not the cash)	Custodies no assets. It books the distribution payable and the accrued platform fee as its own state and instructs the vault to pay them from the sleeve when they are claimed.
LZMinterBurner	None	A stateless dialect translator. It mints and burns on the cross-chain surface it points at and never holds a balance of any kind.

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
FundVault7540	deposit() / mint()	Settlement asset from the subscriber, gross of any entry levy.	Units minted to the receiver.	Any admitted, unfrozen wallet.	The levy stays in the sleeve and is visible only through its own event. With a levy configured and no minimum set, a dust subscription mints zero units and the vault keeps the dust.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
FundVault7540	request Redeem() / requestRedeem WithPolicy()	Units from the owner into the dealing escrow.	None; no cash moves at lodgement.	The owner, an approved operator, or a holder of an ERC-20 share allowance.	The payout locks at the request-time rate. The record retains only the controller, so an operator may direct the resulting claim to any admitted wallet and the owner cannot unwind it.
FundVault7540	cancelRedeem()	None.	Escrowed units returned to the controller.	The request's recorded controller only.	Returns to the controller rather than the owner who supplied the units. Works while paused and after expiry, so units are never trapped, except while that controller is frozen.
FundVault7540	cancelRedeem For()	None.	Escrowed units force-returned into a frozen controller's balance.	Remediation execution role (timelock).	Uses the holding-rule bypass. Scoped to frozen controllers, so it cannot reach units an owner parked under a third party who is not themselves frozen.
FundVault7540	fulfilRedeem()	Escrowed units burned.	No cash yet; the locked payout is ring-fenced as a funded liability.	Settlement role.	Refuses unless the vault already holds every reserved liability plus this payout. Settlement is discretionary, with no ordering rule and no obligation to act before the request expires.
FundVault7540	redeem() / withdraw()	Claimable share and asset balances consumed.	Settlement asset paid to the nominated receiver.	The controller or an approved operator.	Pays the blended locked price of the aggregated bucket. Freeze, redemption pause and lapsed admission all block through the claim limits returning zero, while the cash stays reserved.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
FundVault7540	complete Remediation()	None.	The whole holding moved to the determination address, with unclaimed redemption cash swept there first.	Remediation execution role (timelock).	Supply-preserving. Reads the wallet balance only, so units parked in an open redemption are missed unless the escrow was recovered first, and the freeze is deleted on success.
FundVault7540	rectifyMint() / rectifyBurn()	None.	Units created or destroyed at the named subject.	Rectification role (timelock).	Ordinary legs, so frozen accounts refuse and the escrow is unreachable. No amount bound, no rate window, and the determination reference is accepted even when empty.
FundVault7540	execute()	Whatever the allowlisted leg pulls in.	Whatever the allowlisted leg sends out.	Treasury role.	One address argument may be pinned; amounts never are. Ends in a reserve check, so it cannot spend claim-reserved cash, though a standing unlimited approval lets a counterparty pull outside this path entirely.
FundVault7540	payIncome()	None.	Settlement asset to the distribution or fee payout address.	Income-payer role, granted only to the bound oracle contract.	Reserve-checked after the transfer, so an unfunded distribution reverts and stays booked. A payout address set to zero blocks the claim until governance repoints it.
FundVault7540	laneBurn() / crosschain Burn()	Units destroyed at the named holder.	None.	Lane role, held by the lane proxy.	Accepts an arbitrary holder and an unbounded amount; the deployed lane only ever passes its own caller. No rate window applies, unlike the settlement token's mint rails.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
FundVault7540	laneMint() / crosschain Mint()	None.	Units created at the receiver.	Lane role.	Admission is deliberately not checked, so bridged units land inert under the holding rule. Also bypasses the subscription capacity cap.
FullAsync Vault7540	requestDeposit()	Settlement asset pulled from the owner into the vault.	None; no units are issued.	The owner or an approved operator.	The money is reserved from this instant and refundable. Operator status combined with a standing approval lets a third party move the owner's cash and nominate any eligible controller.
FullAsync Vault7540	cancelDeposit()	None.	Pending subscription cash returned to the owner who paid it.	The owner, the controller, or an operator of either.	Refunds the payer, unlike the redemption side. Blocked while the owner is frozen, and refuses entirely once the application has been struck.
FullAsync Vault7540	fulfilDeposit()	None.	None; reserve-neutral, pending simply becomes claimable.	Settlement role.	Strikes units at the live rate under a publication pin, so a rate post landing mid-run aborts rather than mispricing. Occupies the controller's single open claim until it is collected.
FullAsync Vault7540	deposit() / mint() (claim)	Claimable subscription balance consumed.	Units minted to the nominated receiver.	The controller or an approved operator.	The moment reserved trust money becomes fund property. Mints through a self-call conduit, which is exactly why the treasury path forbids the vault calling itself.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
FullAsyncVault7540	dispose Deposit()	None.	Subscription cash sent to the determination address.	Remediation execution role (timelock).	The only corrective power that refuses to execute without a non-zero determination reference, and the only way to clear a wedged claim slot.
NAVOracle	claimYield() / claimFees()	None.	Settlement asset from the vault sleeve to the distribution or fee wallet.	Distribution role; the timelock for fees.	The oracle never touches cash directly, it instructs the vault. Refuses while pricing is halted, and reverts if the sleeve cannot cover the payment without breaking the reserve.
UnitLane	bridge()	Rail fee token from the caller; units burned on the vault.	A cross-chain message dispatched; the fee consumed by the router.	Any admitted holder.	The burn precedes dispatch atomically. The lane advertises no receiver interface, which production off-ramps probe before delivering, so units can burn here with no corresponding mint on arrival.
UnitLane	ccipReceive()	None.	Units minted to the receiver named in the message.	The configured router only.	Peer-pinned and replay-consumed per delivery key. On a relayer-backed rail the relayer supplies the entire message and is therefore a fully trusted mint authority.
SettlementTokenCore	mint()	None.	Settlement tokens created at any destination.	Minting role.	Destinations are open by election. Bounded by the rail's window, but an unconfigured rail mints without limit and the window resets wholesale, so twice the cap can be issued across a boundary.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Settlement TokenCore	burn() / burnFrom()	Settlement tokens destroyed at a registered treasurer.	None.	Any treasurer, or a spender holding an allowance against one.	Supply may only contract at issuer- registered wallets, so circulating supply cannot silently detach from the reserve the issuer still holds.
Crosschain MintBurn	crosschain Mint() / crosschain Burn()	Settlement tokens created or destroyed at any holder.	The mirror leg on the paired venue.	An authorised bridge grant.	The burn leg carries no treasurer requirement and no rate window, so an authorised bridge is an unbounded destruction authority over every holder's balance.
Settlement TokenCore	forceTransfer()	Settlement tokens moved off a screened holder.	Settlement tokens delivered to a registered treasurer.	Salvage role.	Deliberately bypasses the screening gate while still respecting the pause. The screening and salvage roles are held by the same key in the deployed rail, so one address can screen a vault and then move its balance.
Redemption Router	request Redemption() / request RedemptionWith Permit()	Settlement asset pulled from the holder.	The same amount burned in the call; fiat paid off-chain against the emitted instruction.	Any holder presenting a live issuer signature.	No custody at rest and no partial state. Three independent cut-offs exist: the local pause, revoking the authoriser role, and removing the router's treasurer registration.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of BXCT. We performed our audit according to the procedure described above.

1 Critical, 5 Medium and 4 Low severity issues were found.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses.

Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

September 2026

For

BETTER 

 QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com